# AXLE Planning

The product and plan specialist. Connects what the product is made of with what demand requires, then says what to make, what to buy, by when — and which dates cannot be met.

**Who this is for:** product, engineering, operations and anyone who has to explain XELOR to somebody else.
**What it covers:** the work AXLE reasons about, the rules it cannot break, the exact tools it can call, and what is honestly not built yet.
**Truth standard:** the repository as it stands on 7 August 2026.

AXLE

01 · AT A GLANCE

# What AXLE is for

A role with a fixed tool list, not a general assistant

**In one sentence**

Connects what the product is made of with what demand requires, then says what to make, what to buy, by when — and which dates cannot be met.

| 2 | 2 | 3 | 0 |
|---|---|---|---|
| business areas it speaks for | tools it can actually call | capability families allowed | agents it may delegate to |

### What reaches it

Dated customer demand from MICA · The BOM graph and item policies · Stock and open supply from SPAR · Execution reality from KILN

### What it hands back

Planned orders with their full arithmetic · Exceptions ranked by consequence · A planning work item · A traceable line from demand to proposal

## Capability families it is allowed to touch

`engineering.` `planning.` `agent.`

**Ownership and authority are not the same thing**

The areas above are where AXLE is the right specialist to ask. The tool table on page 8 is the much smaller set it can invoke at runtime. Owning a business area does not grant runtime power over it — and for ONYX, the supervisor, the gap between the two is the widest in the system.

# What sits behind AXLE

Records, screens, workflow, controls and hand-offs

| Engineering | AXLE module · engineering |
|---|---|
| Purpose | Defines the items a factory buys, makes and sells, and the bills of material that explain what a manufactured item consumes. |
| Core records | Item master, item type/UOM/status, GST classification, planning policy links, BOM headers/versions and component quantities/scrap factors. |
| User surface | Items; BOM detail. The current web module is intentionally narrow and mainly exposes the authoritative product structure. |
| End-to-end flow | Create and classify an item; create/version a BOM; validate component references and graph shape; publish the product structure for Planning; snapshot it when Production creates an order. |
| Enforced controls | Typed permissions, tenant RLS, active/versioned BOM lookup and cycle detection when Planning derives low-level codes. |
| Inputs and outputs | Sales selects sellable items, Purchase selects bought items, Planning explodes BOM demand, Production snapshots components and Quality associates specifications/inspections. |
| Current boundary | Item/BOM foundations are live. There is no ECR/ECO request, impact assessment, approval, effectivity or revision audit workflow. |
| Primary code map | `apps/web/src/modules/engineering/manifest.ts · apps/api/src/modules/engineering/` |

### How to read these module pages

“Live” means the records, service rules and user/API surface exist in this repository. It does not mean every surrounding enterprise process is complete. The boundary row names what remains illustrative, manual, simulate-driven or outside the MVP.

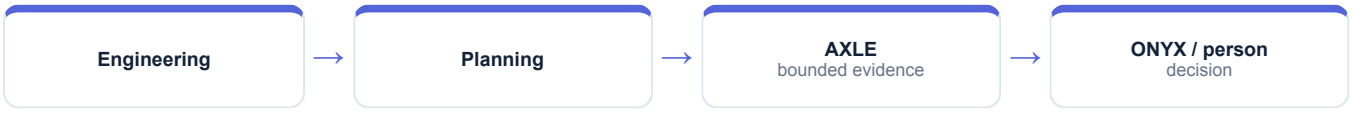
# How AXLE's modules connect

A module owns its records; the agent explains the combined evidence

| Planning | AXLE module · planning |
|---|---|
| **Purpose** | Converts dated demand, stock, open supply, product structure and policy into explainable proposals for what to make or buy and when. |
| **Core records** | Planning policies, calendars, forecasts, demand/consumption, MPS, immutable MRP runs/workings, planned orders, pegging, exceptions, requisitions, capacity views and draft schedules. |
| **User surface** | MRP; Planned orders; Exceptions; Demand; Policies; Explain. |
| **End-to-end flow** | Compute low-level codes; consume forecast with nearby orders; net demand bucket by bucket; apply safety stock and lot-sizing; offset over working days; explode components into release buckets; persist workings/pegs; create ranked exceptions; optionally firm or convert a proposal. |
| **Enforced controls** | Cycle refusal, required work calendar, immutable completed-run workings, upward lot rounding, past-due flagging, plan-to-execution uniqueness and explicit exception acceptance/snooze behavior. |
| **Inputs and outputs** | Reads Sales demand, Engineering BOMs, Inventory balances, Purchase open supply and Production state; outputs buy/make proposals to Purchase and Production. |
| **Current boundary** | MRP, pegging and explainability are live. Capacity is an infinite-capacity report, scheduling is a draft heuristic, and firmed orders do not yet feed the next run reliably. |
| **Primary code map** | `apps/web/src/modules/planning/manifest.ts · apps/api/src/modules/planning/` |

### Cross-module coordination

Engineering defines what a product is; Planning applies demand, stock, supply, calendars and policy to decide what must happen by date. Production receives proposals and a pinned structure, so later master-data changes cannot rewrite work already released.



# How AXLE's world actually runs

The business pipeline, not the agent plumbing

One MRP run, eight steps. It is a pure function: the service gathers inputs through ports, calls the engine, and stores the answer WITH its working — because the question asked in November is always "why did we buy fifty castings in July?"

1 **Levels first**
Low-level codes are derived by topological sort. An item netted at the wrong level is netted before its demand exists. A cycle is refused outright, naming the loop in item codes rather than database ids.

2 **The calendar**
Every lead time is offset over a working-day calendar — Mon–Sat by default. No calendar, no run: a two-week lead time quoted by a foundry means twelve working days, not fourteen.

3 **Demand**
Per bucket, demand = max(forecast, orders), after an order has consumed the forecast nearest to it in time. A line with no requested date lands in the FIRST bucket with a warning, because burying it at the end of the horizon would hide it.

4 **Netting**
Item by item, up the levels: Net = Gross − Scheduled − Available(t−1) + Safety Stock. Safety stock is a floor the plan refuses to eat into, not a buffer it may borrow from.

5 **Lot sizing**
Six rules, and every one of them rounds UP. A rule that can return less than the shortfall has quietly turned a sizing policy into a stockout.

6 **Offsetting**
The release date is walked backwards over working days. A date already in the past is clamped to today and FLAGGED — never back-dated, because a planned order dated last Tuesday is a lie that makes the horizon look feasible.

7 **Explosion**
Components land in the parent's RELEASE bucket, not its receipt bucket — they are needed when it starts, not when it finishes. Scrap is a gross-up rather than a markup: 5% scrap divides by 0.95, it does not multiply by 1.05.

**How much of this AXLE can actually see**

Of everything above, AXLE can read exactly one thing directly — planned orders. The rest of this pipeline is context for judging what it reads; it is not data the agent can reach. Where a mission needs more, it asks the specialist who owns it or it says the evidence is missing.

04 · THE RULES UNDERNEATH

# What the code will not let anyone do

Enforced by constraints and locks, not by wording

### A completed run is immutable

The per-bucket arithmetic is the answer to why something was bought. Triggers refuse UPDATE and DELETE on the workings, and a completed run cannot be reopened — make a new one.

### A plan cannot be ordered twice

A unique index ties one planned order to one execution document. The application check is only the friendly error; the database is what actually prevents it.

### MRP and reorder point cannot both hold an item

A CHECK constraint forbids it outright — it is the commonest silent source of excess inventory.

### Accepting an exception does not perform it

The reply is literally “do that next; accepting did not do it for you”. A snooze without a date is a dismissal wearing a friendlier word, and is refused.

**Why this page matters more than the agent pages**

An agent is only as trustworthy as the system it reads. Every rule here holds whether the request came from a person, a screen, a seeder or AXLE — which is precisely why an agent can be given a read tool without also being given a way to do damage.

## Where these rules are kept

```
packages/platform/src/planning/netting.ts
```

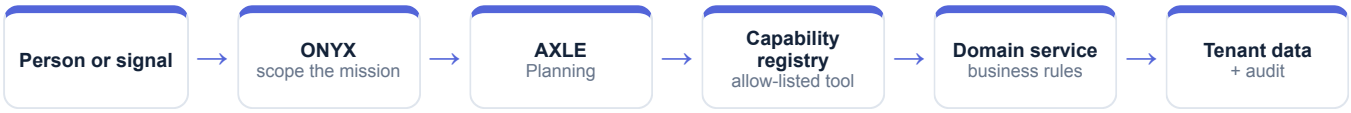
```
apps/api/src/modules/planning/mrp.service.ts
```

```
packages/platform/src/planning/exceptions.ts
```

05 · RUNTIME CONTRACT

# How AXLE takes part in a mission

The engine controls order, parallelism and recovery




## 1 · Scope

ONYX turns the goal into a run against a frozen graph version, with a fixed tenant, user, step budget and timeout.

## 2 · Authorise

Two checks, both required: the tool must be on AXLE's allow-list, and the signed-in person must independently hold the permission.

## 3 · Execute

A registered domain service does the work under the same rules a screen would face. There is no general SQL doorway.

## 4 · Preserve

Inputs, outputs, events and a checkpoint after every wave, so the run can be explained or resumed.

**An agent can narrow authority, never widen it**

AXLE is a specialist and delegates to nobody. Because the permission check is repeated against the requesting person, a mission can never reach data that person could not have opened themselves.

06 · EXACT CALLABLE SURFACE

# Everything AXLE can call

This table is the complete list — there is no other door

| Capability | Mode | What comes back | Permission required | Needs approval |
|---|---|---|---|---|
| planning.planned-orders.read | Read | Planned orders | planning.mrp.read | No |
| agent.action.dispatch | Execute | Governed work item | agentos.run.operate | Yes |

### Read · Analyse · Simulate

Return evidence or a reproducible view. Nothing in the business changes.

### Draft

Produce reviewable content that is labelled a draft and can never present itself as an approved outcome.

### Execute

The only side-effecting mode in the whole system, and the only one that requires an approved human gate above it.

### Both locks must open

The agent's allow-list AND the person's permission. Either one failing is a refusal.

**What “dispatch” does and does not do**

It appends a governed work item naming the responsible specialist, the approval it came from and the exact payload. It does not change a sales order, a stock balance, a production order, a journal or a customer message — a person still does that work.

07 · IN THE SIX MISSIONS

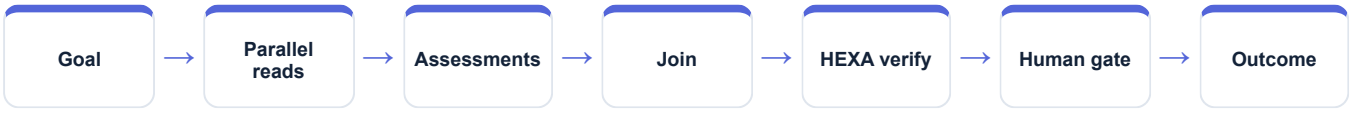
# Where AXLE actually appears

Node by node, from the registered graph definitions

| Mission graph | What AXLE does in it |
|---|---|
| **Cross-functional readiness** | Not involved |
| **Nine-agent operating review** | Reads planned orders, then the planning assessment |
| **Controlled action mission** | Reads, recommends, and dispatches the capacity scenario |
| **Working Capital Review** | Not involved |
| **QMS & Audit Readiness** | Not involved |
| **Managed Service Assurance** | Not involved unless a service issue requires plan evidence |

> **Reading this honestly**
>
> “Not involved” is not a limitation, it is the design. A mission asks only the specialists whose evidence it needs, and a specialist cannot add itself to a graph at runtime. The graph is written down, versioned and content-hashed before the run starts.

## The shape every mission shares



08 · WORKED EXAMPLE

# What the plan says has to happen

Real records from the running demo, not an illustration

PMP-PX400 is an impeller, a shaft, a casing, two cartridge seals and sixteen bolts — and the impeller and shaft have their own BOMs underneath, so the explosion runs two levels deep.

120 pumps pull roughly 707 kg of 316L through those two levels. The bar has a sixteen-working-day lead time and a 250 kg minimum, which is why the exception exists at all.

The run shows its work per bucket, in words: “120 wanted − 40 already coming − 15 on hand + 10 safety stock = 75 short.” A planner accepted one exception with a note that Meridian confirmed the balance heat.

**The sentence to say out loud**

“AXLE contributes evidence from its own area, with the source records behind it and its limits stated. It does not claim an action is done until the system has recorded and verified it.”

## Fair questions to ask while watching

- Which records is this answer standing on?
- Which permission was checked, and against whom?
- Is this a recorded fact, a simulation, a draft, or a completed result?
- Would this step have waited for a person?
- What happens if the service restarts halfway through?

09 · WHAT IS REAL

# Live today, and deliberately not

The second column is the one worth reading

### Working now

- Multi-level netting with pegging back to the originating order
- Low-level codes and cycle detection
- Six lot-sizing rules and working-day offsetting
- Seven exception types ranked by consequence
- Immutable run workings and an explain-in-words view

### Not built — stated plainly

- No ECR/ECO change control — a BOM records the result of a change, never the change
- MRP is infinite-capacity; the capacity pass is a report that changes nothing
- Finite scheduling is a labelled heuristic and always lands as a draft — it never publishes itself
- Firming a planned order does not yet feed it back into the next run

| | |
|---|---|
| **Identity boundary** | Verified token → tenant and actor |
| **Authority boundary** | Agent allow-list AND the person's permission |
| **Action boundary** | Side effects need an approved ancestor node |
| **Data boundary** | Domain service over a tenant-fenced database |
| **Evidence boundary** | Append-only runs, events, checkpoints and audit |

Gaps are listed because a guide that hides them fails the first hour of technical due diligence. Every item on the right is either a roadmap decision or a known defect, and both are recorded in the project's own capability-gap register.

10 · QUICK REFERENCE

# AXLE on one page

For explaining the agent to somebody new

### Its job

Connects what the product is made of with what demand requires, then says what to make, what to buy, by when — and which dates cannot be met.

### Speaks for

Engineering, Planning

### Takes in

Dated customer demand from MICA · The BOM graph and item policies · Stock and open supply from SPAR · Execution reality from KILN

### Gives back

Planned orders with their full arithmetic · Exceptions ranked by consequence · A planning work item · A traceable line from demand to proposal

### Can call

planning.planned-orders.read, agent.action.dispatch

### Cannot do

No ECR/ECO change control — a BOM records the result of a change, never the change · MRP is infinite-capacity; the capacity pass is a report that changes nothing · Finite scheduling is a labelled heuristic and always lands as a draft — it never publishes itself

## Words used on these pages

| | |
|---|---|
| **Agent** | A named specialist with a fixed, allow-listed set of tools — not a process that can roam. |
| **Capability** | One registered operation with a declared mode, owner and required permission. |
| **Mission graph** | A versioned recipe: which steps run, which run together, where it waits for a person. |
| **Checkpoint** | A stored snapshot after each wave, used to explain a run and to resume it safely. |
| **Governed work item** | An approved, append-only assignment for a human team — not the business change itself. |